LEVEL-4 SYNCHRONOUS FIFO

Complete Working Explanation

`Module: robust_sync_fifo` | `Testbench: tb_robust_sync_fifo`

Explicit acceptance policy for simultaneous read and write operations

1. Introduction

A **synchronous FIFO** (First-In First-Out buffer) is a small memory that stores data words and returns them in the same order they arrived. Both the write side and the read side run on the **same clock** (`clk`), which keeps the design simple and free of clock-domain-crossing problems.

The **Level-4** version of this FIFO adds one important idea: a clearly defined **acceptance policy**. A request on `wr_en` or `rd_en` is only a *request*. The FIFO decides, using its current state, whether the request is **accepted** (`wr_accept`, `rd_accept`). This matters most when read and write are requested on the same clock edge while the FIFO is full or empty.

Property	Value in this design
Data width	8 bits per word (`DATA_WIDTH = 8`)
Depth	4 words (`DEPTH = 4`), addressed by 2-bit pointers
Occupancy counter	3 bits (`COUNT_WIDTH = 3`), counts 0 to 4
Clocking	Single clock, rising edge; active-high synchronous reset
Read output	Registered `data_out`: valid one clock after an accepted read
Level-4 feature	Explicit `wr_accept` / `rd_accept` outputs and a fixed rule for simultaneous read/write at full and empty

2. Architecture

The FIFO is built from four cooperating blocks (shown in Figure 1): a **memory array**, a pair of **pointers** (write and read), an **occupancy counter**, and the **status / acceptance logic** derived from that counter.

2.1 Parameters

Parameter	Default	Meaning
DATA_WIDTH	8	Width of each stored word
DEPTH	4	Number of storage locations
ADDR_WIDTH	2	Bits needed to address DEPTH locations (log2 of 4)
COUNT_WIDTH	3	Bits needed to hold values 0 to DEPTH (0 to 4 needs 3 bits)

2.2 Block diagram

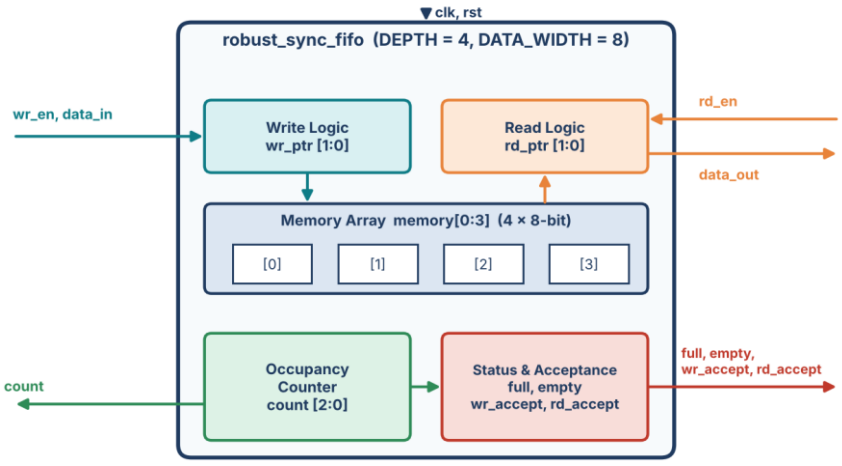


Figure 1: Block-level architecture of robust_sync_fifo

2.3 Ports

Port	Dir	Width	Description
clk	Input	1	Common clock for all logic (rising edge)
rst	Input	1	Active-high synchronous reset
wr_en	Input	1	Write request
data_in	Input	DATA_WIDTH	Word to be written
rd_en	Input	1	Read request
data_out	Output	DATA_WIDTH	Registered read data (holds last value when no read is accepted)
full	Output	1	High when `count == DEPTH`
empty	Output	1	High when `count == 0`
count	Output	COUNT_WIDTH	Number of words currently stored
wr_accept	Output	1	High when a write is actually performed
rd_accept	Output	1	High when a read is actually performed

3. Internal Working, Block by Block

3.1 Memory array

```
RTL
reg [DATA_WIDTH-1:0] memory [0:DEPTH-1];
```

Four 8-bit registers hold the data. The array is **not reset**: old words simply stay in the slots until they are overwritten. The FIFO never needs to clear them, because the counter and pointers decide which slots are valid.

3.2 Read and write pointers

```
RTL
reg [ADDR_WIDTH-1:0] wr_ptr;   // next slot to write
reg [ADDR_WIDTH-1:0] rd_ptr;   // next slot to read
```

- `wr_ptr` always points at the **next free slot**. After a write it moves forward by one.
- `rd_ptr` always points at the **oldest stored word**. After a read it moves forward by one.
- Both pointers **wrap** from slot 3 back to slot 0, so the four slots behave like a **circular buffer**.

3.3 Status flags: full and empty

```
RTL
assign full  = (count == DEPTH);
assign empty = (count == 0);
```

The flags come directly from the occupancy counter. There is no pointer comparison, so there is no ambiguity between the full and empty cases.

3.4 Acceptance logic (the Level-4 core)

```
RTL
assign wr_accept = wr_en && !full;
assign rd_accept = rd_en && !empty;
```

This is the heart of Level 4. Each accept signal depends **only on the registered state** (`full`, `empty`), never on the other request. That is why a write is refused at full and a read is refused at empty, even when the opposite operation is requested at the same time.

3.5 Write path

```
RTL
always @(posedge clk) begin
    if (rst)
        wr_ptr <= 0;
    else if (wr_accept) begin
        memory[wr_ptr] <= data_in;          // store the word
        if (wr_ptr == DEPTH-1) wr_ptr <= 0; // wrap
        else wr_ptr <= wr_ptr + 1'b1;       // advance
    end
end
```

On each rising edge, if the write is accepted, `data_in` is stored at `memory[wr_ptr]` and the pointer advances. If the write is not accepted, nothing changes: the data is **silently dropped**, and `wr_accept` stays low so the outside logic can see it.

3.6 Read path

```
RTL
always @(posedge clk) begin
    if (rst) begin
        rd_ptr   <= 0;
        data_out <= 0;
    end
    else if (rd_accept) begin
        data_out <= memory[rd_ptr];         // register the word
        if (rd_ptr == DEPTH-1) rd_ptr <= 0; // wrap
        else rd_ptr <= rd_ptr + 1'b1;       // advance
    end
end
```

The read is **registered**: the word appears on `data_out` one clock edge after the read is accepted. When no read is accepted, `data_out` **keeps its previous value**.

3.7 Occupancy counter

```
RTL
case ({wr_accept, rd_accept})
    2'b10: count <= count + 1'b1;   // write only
    2'b01: count <= count - 1'b1;   // read only
    2'b11: count <= count;          // both: net zero
    2'b00: count <= count;          // neither
endcase
```

The counter is updated using the **accepted** signals, not the raw enables. This is what keeps the count correct in every corner case.

wr_accept	rd_accept	count update	Reason
1	0	**count + 1**	One word enters
0	1	**count - 1**	One word leaves
1	1	**unchanged**	One enters, one leaves
0	0	**unchanged**	Nothing accepted

4. Level-4 Acceptance Policy

The design header states four rules. They are shown here with the signal-level reason each one holds.

Rule	Statement	Why it holds in the RTL
1	Write is accepted only when the FIFO is **not full**	`wr_accept = wr_en && !full`
2	Read is accepted only when the FIFO is **not empty**	`rd_accept = rd_en && !empty`
3	At **full**, a simultaneous read does **not** allow a write on that same edge	`full` is 1, so `wr_accept` is 0 whatever `rd_en` is. The read frees a slot, but a write can only use it on the **next** edge.
4	At **empty**, a simultaneous write does **not** allow a read on that same edge	`empty` is 1, so `rd_accept` is 0 whatever `wr_en` is. The write stores a word, which can be read from the **next** edge.

4.1 Complete behaviour table

FIFO state	wr_en	rd_en	wr_accept	rd_accept	count change	Outcome
Empty	1	0	1	0	**+1**	Normal write
Empty	0	1	0	0	**0**	Read ignored (nothing to read)
Empty	1	1	1	0	**+1**	**Rule 4:** write accepted, read blocked
Partly filled	1	0	1	0	**+1**	Normal write
Partly filled	0	1	0	1	**-1**	Normal read
Partly filled	1	1	1	1	**0**	Both accepted, count unchanged
Full	1	0	0	0	**0**	Write dropped (overflow prevented)
Full	0	1	0	1	**-1**	Normal read
Full	1	1	0	1	**-1**	**Rule 3:** read accepted, write blocked

Why use this policy?

The accept signals depend **only on registered state**, so there is no combinational path from `rd_en` into `wr_accept` (or the reverse). Timing stays simple and the behaviour at the boundaries is fully predictable.

The cost is that at full (or empty) the FIFO gives up one possible transfer. The other side's request is handled normally on that edge, and the blocked request can be accepted on the next edge.

5. Circular Buffer Operation

The four memory slots form a ring. The write pointer (**WR**) and read pointer (**RD**) chase each other around it. Green slots hold **valid** data (the `count` slots starting at RD). Grey values are **stale** words that were already read and will be overwritten later.



Figure 2: Pointer and count movement during the testbench sequence

Because the counter, not the pointers, tells full from empty, `wr_ptr == rd_ptr` is never ambiguous. It can mean empty (count 0) or full (count 4), and `count` always says which.

5.1 Reset behaviour

Item	Value after reset
wr_ptr, rd_ptr	0
count	0 (so `empty = 1`, `full = 0`)
data_out	0
memory[0..3]	Not cleared (contents are ignored until overwritten)

6. Testbench: Cycle-by-Cycle Walkthrough

The testbench generates a 10 ns clock (rising edges at 5, 15, 25 ... ns) and changes its inputs on the **falling edge**, so inputs are stable well before each active edge. The tables below follow the exact sequence in `tb_robust_sync_fifo.v`.

Edge (ns)	Inputs set before edge	Action at edge	wr_ptr	rd_ptr	count	data_out / memory
5	rst = 1	Reset	0	0	0	data_out = 00
15	rst = 0, no requests	Idle	0	0	0	No change
25	wr_en = 1, data_in = 10 (set at 20 ns)	Write 10	1	0	1	memory[0] = 10
35	wr_en = 1, data_in = 20 (set at 30 ns)	Write 20	2	0	2	memory[1] = 20
45	wr_en = 0, rd_en = 1 (set at 40 ns)	Read	2	1	1	data_out = 10
55	wr_en = 1, data_in = 30, rd_en = 1 (set at 50 ns)	Read + Write	3	2	1	memory[2] = 30, data_out = 20
65	wr_en = 0, rd_en = 1 (set at 60 ns)	Read	3	3	0	data_out = 30

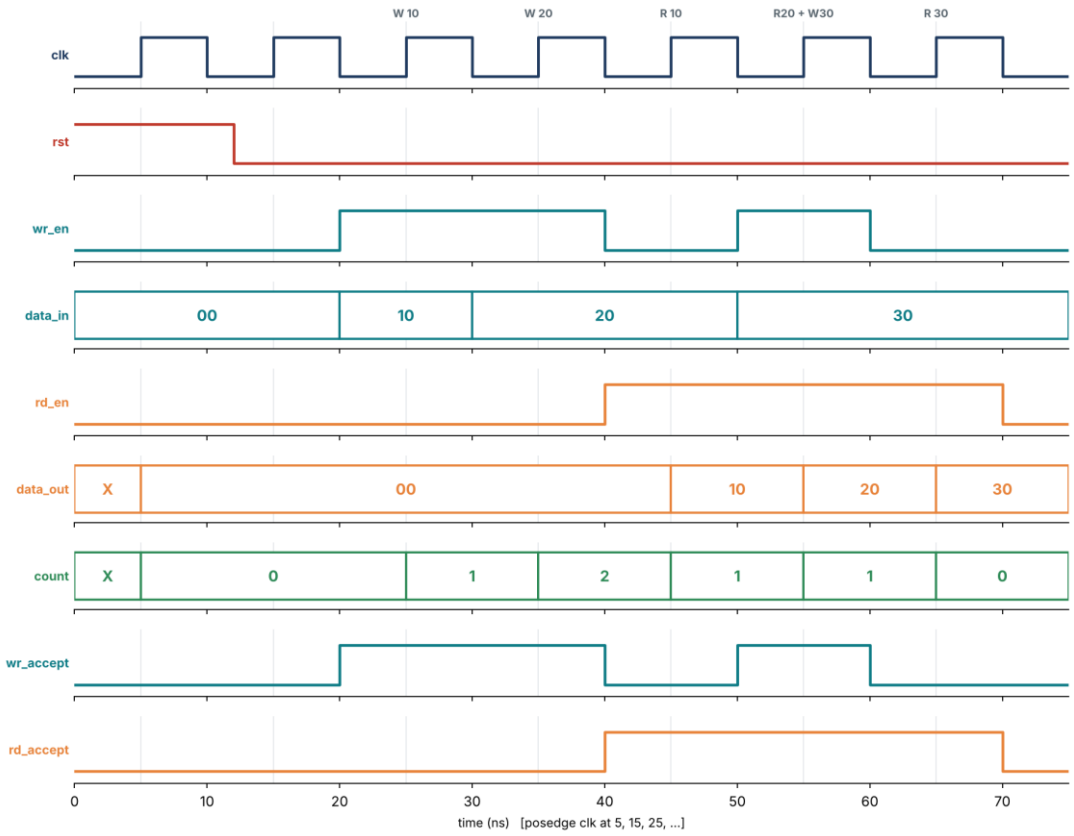


Figure 3: Waveform of the testbench sequence (derived by tracing the RTL)

6.1 Checks performed by the testbench

Time	Test	Check	Expected	Result
50 ns	Read ordering	`data_out !== 8'h10`	10 (oldest word first)	**PASS: data_out = 10**
60 ns	Simultaneous op	`count != 1`	count = 1 (unchanged)	**PASS: count = 1**
70 ns	Empty condition	`!empty`	empty = 1	**PASS: count = 0**

All three checks pass, so the testbench prints **LEVEL 4 PASS** and ends with `$finish`. Any failed check would call `$fatal` and stop the simulation.

7. Coverage Notes and Extra Tests

The supplied testbench confirms FIFO ordering, the simultaneous read/write case in the middle of the FIFO, and the empty flag. It does **not** yet reach the two boundary rules that make this design Level 4 (Rules 3 and 4), and it never drives the FIFO full. The tests below would close that gap. Expected values are from the RTL logic above.

#	Stimulus	Check	Expected result
A	Write 4 words, then set `wr_en = 1` with `data_in = 55`	`full`, `count`, `wr_accept`	full = 1, count = 4, and the 5th write is dropped (`wr_accept = 0`)
B	While full, set `wr_en = 1` and `rd_en = 1` together	`wr_accept`, `rd_accept`, `count`	wr_accept = 0, rd_accept = 1, count becomes 3 (Rule 3)
C	While empty, set `wr_en = 1` and `rd_en = 1` together	`wr_accept`, `rd_accept`, `count`	wr_accept = 1, rd_accept = 0, count becomes 1 (Rule 4)
D	While empty, set only `rd_en = 1`	`rd_accept`, `data_out`, `count`	rd_accept = 0, data_out keeps its old value, count stays 0
E	Write and read more than 4 words in total	Pointer wrap-around	Words still come out in order after wr_ptr and rd_ptr wrap from 3 to 0

8. Summary

Key points

Storage: 4 x 8-bit memory used as a circular buffer with `wr_ptr` and `rd_ptr`.

Status: `count` tracks occupancy; `full = (count == 4)` and `empty = (count == 0)`.

Acceptance: `wr_accept = wr_en && !full` and `rd_accept = rd_en && !empty`.

Level-4 policy: at full, read wins and the write waits; at empty, write wins and the read waits.

Counter: updated from the accepted signals, so rejected requests never corrupt the count.

Read timing: `data_out` is registered, giving one clock of read latency.

Testbench result: the supplied test sequence passes (LEVEL 4 PASS); adding tests A to C would also verify the full and empty boundary rules.